

Project Context: Artemis Lookalike Domain Detection Engine

This file provides project-level context for AI agents working in this repository via Google Antigravity. Read this before planning any work that touches domain-risk scoring, the prediction pipeline, or the frontend dashboard that displays its results.

What this project does

Artemis is a machine-learning engine that detects typosquatting, homoglyph attacks, and other lookalike domains. It combines:

- An **ExtraTreesClassifier** trained on lexical/structural domain features
- A **deterministic brand-matching algorithm** (weighted similarity)
- Three **external domain intelligence sources**: Tranco, Majestic Million, Cisco Umbrella

Core capabilities agents should be aware of when reasoning about this codebase:

1. **ML Risk Classification** - probability that a domain is a lookalike
2. **Brand Identification** - which real brand is being targeted
3. **Explainability** - 29 lexical features (entropy, edit distance, etc.) plus a named attack technique (e.g. "Character Insertion")
4. **Domain Intelligence** - cross-reference against top-1M traffic databases

Environment & dependencies

Install from `requirements.txt` before running or testing anything that touches the model:

- `pandas`, `numpy`, `scikit-learn` — ML processing/pipeline
- `tldextract` — domain parsing
- `Levenshtein`, `rapidfuzz` — string similarity
- `joblib` — model loading

The `data/` folder must contain the Tranco, Majestic, and Cisco CSVs for Domain Intelligence features to work. If an agent is asked to run predictions and this folder is missing or empty, it should flag that rather than silently returning partial/degraded results.

How to call the model

Do **not** shell out to a CLI. Call the prediction function directly from backend code (FastAPI/Django/Flask):

```
python
```

```
import sys
import os
sys.path.append(os.path.join(os.path.dirname(__file__), 'lookalike_model', 'src'))
from predict import predict_domain

try:
    result_payload = predict_domain(
        "paypal.com",
        model_path="models/lookalike_domain_model.joblib"
    )
except Exception as e:
    print(f"Error processing domain: {e}")
```

`predict_domain` returns a Python dict that serializes directly to JSON for the frontend.

Output schema

json

```

{
  "domain": "paypall.com",
  "impersonated_brand": "paypal",
  "brand_score": 0.88,
  "probability": 0.92,
  "risk": "HIGH",
  "technique": "Character Insertion",
  "target_domain": "paypal.com",
  "features": {
    "domain_length": 11,
    "entropy": 2.914,
    "levenshtein_distance": 1,
    "jaro_winkler_similarity": 0.982,
    "digit_count": 0,
    "hyphen_count": 0,
    "subdomain_count": 0,
    "...": "(22 other numeric features)"
  },
  "intelligence": {
    "tranco": { "match_type": "SIMILAR", "matched_domain": "paypal.com", "ran
    "majestic": { "match_type": "NONE", "matched_domain": null, "rank
    "cisco": { "match_type": "SIMILAR", "matched_domain": "paypal.com", "ran
    "agreement": "2 / 3",
    "agreed_domain": "paypal.com"
  }
}

```

Field semantics agents must respect

Core metrics

- `probability` — float 0.0-1.0. `> 0.5` → LOOKALIKE, else LEGITIMATE. Display as a percentage.
- `risk` — enum: `"HIGH"` (prob $\geq$ 0.8), `"MEDIUM"` (prob $\geq$ 0.5), `"LOW"` (prob $<$ 0.5). Used for color-coding.
- `impersonated_brand` — string or `None`.
- `technique` — string, e.g. `"Character Insertion"`, `"Homoglyph / Character Substitution"`.

Explainable features (`result["features"]`)

- Pass through to the frontend for an "Evidence" / "Model Explanation" panel.
- Highest-priority fields to surface: `entropy`, `levenshtein_distance`, `domain_length`, `digit_count`, `hyphen_count`.

Domain intelligence (`result["intelligence"]`)

- Sub-dicts for `tranco`, `majestic`, `cisco`.
- `match_type` enum: `"EXACT"`, `"SUBDOMAIN"`, `"SIMILAR"`, `"NONE"`.
- `agreement` — string like `"3 / 3"`, `"2 / 3"`, `"NO"` — cross-dataset consensus.

Frontend / UI rules agents must follow

When an agent builds or edits any UI surface for this engine, it must adhere to these constraints (do not deviate without explicit user instruction):

1. **Strict separation** — ML predictions (`probability`, `risk`, `impersonated_brand`) are a distinct visual section from Domain Intelligence (`tranco`/`majestic`/`cisco`). These are two different kinds of evidence (model inference vs. database presence) and should never be merged into one card or table.
2. **Explainability first** — never just show a verdict like "Malicious." Always expose the `features` dict (e.g. in a side panel or tooltip) so the reasoning ("Edit distance: 1", "Entropy: 3.9") is visible.
3. **Color semantics:**
 - `risk: "HIGH"` → error/red
 - `risk: "LOW"` → success/green
 - Domain Intelligence matches (Tranco/Majestic/Cisco) must **not** default to green/"safe" — an exact match in a top-traffic DB does not mean the domain is safe (it can be compromised). Treat these as neutral/contextual (grey or blue), never as a safety signal on their own.

Guidance for agents integrating this into the larger app

- Treat `predict_domain(...)` as the single integration point into the ML/intelligence layer — wrap it in a thin service function rather than importing internals from `lookalike_model/src` elsewhere in the codebase.
- Any new backend route that exposes this should return the payload close to the schema above rather than reshaping it, so the frontend contract stays stable.
- If asked to add caching, batching, or async handling around `predict_domain`, preserve the exception-handling shown above — failures per-domain should not take down a batch request.
- When generating tests or fixtures, use the `paypall.com` example above as the canonical golden-path fixture.